Project 2: Maintenance Documentation

Development Environment

- Programming Language: Python 3.13.15
- Game Library: Pygame
- Pygame must be installed to run UI.py

Estimated Person-Hours:

Team Member	Assigned Task	Estimated Hours	Actual Hours
Atique	Coding	2	6
Viren	Coding	3	3
Ever	Coding	7	6
Aidan	Documentation	2	2
Parker	Documentation	2	2
Joseph	Documentation	4	3

Estimation Methodology

Each team member was assigned their task of either coding or documentation during the 4th group meeting and asked for a general estimate of how long they believed their responsibilities would take to complete. We tracked hours in a separate currently-working chat on our discord to determine actual hours.

System Additions Overview:

Added Components:

Ai_solver.py:

- AISolver class has 3 difficulties implemented; easy, medium, and hard. The user is allowed to pick whether they want the AI to solve the minesweeper board live while the game is running as well as which difficulty setting they want used. There is also an “interactive” mode where the user and AI will take turns clicking on singular tiles until the puzzle is solved.
    - easy_move
        - Difficulty rules

        - The AI uncovers cells completely randomly without additional logic in ‘solver’ mode. When used in ‘interactive’ mode, the AI will not attempt to click on a tile that has already had its number revealed or a flag placed on top of it.
    - Code logic
        - The only logic used for this function is the check for covered tiles, it is otherwise completely random and not very good at solving puzzles without extreme luck.
- medium_move
    - Difficulty rules
        - The AI follows 2 rules in order. First, the AI checks if the number of empty tiles around its ‘current’ tile is equal to the number of remaining mines. If yes, it will flag each empty tile as having a mine.
        - Next, the AI will check if the number of its ‘current’ tile matches the number of flags that surround it. If yes, the AI will uncover each of the tiles around it.
        - If neither of the above rules apply, the AI will click on a random tile (using the same ruleset as easy difficulty)
    - Code logic
        - The function scans the entire board for its uncovered number tiles.
        - When a number tile is found, the function will examine its 8 neighbor tiles and save the state of them to two possible lists: ‘hidden_neighbors’ and ‘flagged_neighbors’
        - The function then compares the number of hidden and flagged neighbors to the rules listed above and makes the appropriate move. If neither above rule applies, it will pick a tile at random similar to how the easy difficulty works.
- hard_move
    - Difficulty rules
        - Hard mode follows all of the same logic as described in the medium section with respect to the ‘1-2-1’ rule. The solver will automatically place flags on the two corner tiles of a 2 tile if it is placed between two 1 tiles.
    - Code logic
        - Call a separate ‘_apply_121’ function that check for usage of the 1-2-1 rule. The function first checks for any horizontal instances of the pattern and then vertical instances if no horizontal one is found.

- The function will place flags on the corner tiles of the 1-2-1 pattern if one is found. If no patterns are found at all, pass on to the next step.
- If no 1-2-1 pattern is found, call the ‘medium_move’ function for simplicity (since they follow the same rules).
- If ‘medium_move’ doesn’t find any valid moves, the AI will fallback to doing a random move similarly to the easy AI mode.

game_timer.py:

- Implements the GameTimer function. The function is relatively simple, using the ‘time’ import. This lets the code use a simple clock function to return the time that has elapsed since the first tile was clicked on second by second.

UML Diagram:

